

CSE 425: Python Tutorial [incomplete] For Beginners

Md. Shahriar Karim

Associate Professor

Department of Electrical and Computer engineering

North South University, Bangladesh

Email: shahriar.karim@northsouth.edu

Copyright: ©Md. Shahriar Karim, NSU

Contents

1	Introduction	3
2	Python: convention and syntax	4
3	Basics of variables and functions	4
3.1	Variables	4
3.2	Defining a Function in Python	5
3.2.1	Example: Function classifying the sign of a number	5
3.2.2	Example: Calculate circle area by defining a function	6
3.2.3	Example: Plot Mathematical Functions	7
3.3	Concepts of Higher Order Functions	7
3.3.1	Example HOF and List-comprehension: Map	9
3.4	Strings: Slicing	10
3.4.1	String operations	10
4	Control statements	10
4.1	If-else and Nested If-else	11
4.2	While Loop	12
4.3	For Loop	12
5	Data structures in Python	13
5.1	Lists	13
5.2	Tuple	17
5.3	Strings	18
5.4	Sets	19
5.5	Dictionaries	20
6	List Comprehension	21
6.1	Quicksort algorithm using list comprehension	22
7	Object Oriented Programming (OOP)	22
7.1	Objects	22
7.2	Class Definition	23
7.3	Define Classes and Objects in Python	24
7.4	<code>__init__()</code> in Python: Constructor for a Class	24
8	Concurrency and Parallelism	25
8.1	Multiprocessor Architecture	25
8.2	Concurrency	25
8.3	Parallelism	26
9	Numerical Simulation	26
9.1	Review of Linear Algebra	26
9.1.1	Vector	27

Disclaimer: This is a working document developed as part of the course "Concepts of Programming Language, CSE 425", made available in its current **incomplete** form only to facilitate students in their in-class learning and practices on Python-basics.

1 Introduction

The concepts of data types in programming languages have evolved over the past 60 years. In the early years, only limited data structures were supported with which the other data types were implemented. As the language evolution continues, many data types are now accommodated through inbuilt commands or definitions. For instance, Python provides numerous forms of data types along with various libraries and other facilities for data science applications. **[to be completed]**

Python highlights

- Python is **easy to learn** and widely applicable programming language. It provides several high-level data structures, making it suitable for data-extensive analysis in many areas. Python also provides a simpler yet efficient approach of **object-oriented programming**.
- Python is an interpreter-based programming language. If started, the interpreter presents **>>>** symbol to the user as the prompt for writing commands.
- **Dynamically typed:** variables can hold alternative forms of values as the code executes. That is, variables can take any data type during execution. **In contrast**, variables are statically typed in C-programming. Interestingly, a fixed type, size, and location in memory are assigned immediately after the assignment statements in C-programming.
- Python extensively uses the concept of **list** as the compound data types, which can be compared with the **array** types as frequently used in other programming languages.
- Download the interpreter and standard libraries from <https://www.python.org/>.

Python provides better readability and concise code syntax. The program written in Python is often shorter and more readable as compared to programs written in C, C++, or Java. A few key reasons that contributed to such improvement in readability are as follows:

- No variable or argument definition is necessary. A user can use a variable immediately after its name is chosen.
- Indentation handles code blocks or grouping of statements during different flow-control statements.
- High-level data types allow expressing complex programming tasks even in lines. For instance, the concept of **list comprehension** achieves complex operations in a single line

and forms the target data as needed. Also, the concept of **higher order function (HOF)** additional facility for data processing.

- For online code execution visualization and debugging, you may wish to use <https://pythontutor.com/>

2 Python: convention and syntax

Python is an interpreter-based programming language that offers straightforward syntaxes for various language construct [1, 2]. For instance, the traditional symbols and operators, such as `+`, `-`, `*`, and `/` in other previous programming languages, remain of the same operational meaning in Python.

3 Basics of variables and functions

3.1 Variables

As in C programming, variables' types should be defined before they can be assigned a value. In contrast, the **Type definition is not needed in Python code**; a variable can be used immediately without its type definition.

```
x = 7 # data type not mentioned, but the assignment is still valid.
y = 20
z = x + y
print('The sum of %d and %d is: %d'%(x, y, z))
type(x)
type(y)
type(z) # Integer type
```

Again, the same **x (integer valued)** can have **string type** elements assigned:

```
x = 7
x = "Hello World"
type(x) # Str type
```

Precisely, **type-checking in Python occurs dynamically**. That is, when any operation uses the defined variable, say *x* in this case, Python does the type checking to decide if the operation is legal or illegal. For instance:

```
x = 1.2 # float data
len(x)
TypeError: object of type 'float' has no len()

x = 7 # Integer (int) type data
len(x)
TypeError: object of type 'int' has no len()

x = "Hello World"
```

```
len(x)
Output: 11
```

As observed, the `len()` measures the length of the string type values and generate error when applied for integer and float type. Interestingly, Python checks the type dynamically when the `len()` operation is performed.

3.2 Defining a Function in Python

In Python, functional definition looks significantly similar to C. A few key points are necessary for functional definition in Python are as follows:

- a) Definition of a function starts with `def`, followed by a `function name` and `':'`
- b) Type definition for return type is not necessary.

Function definition: example

```
def function_name (input):      # definition starts with def
    evaluate = expression of inputs
    return evaluate
Example:
def test_fnc (x): # x is input
    evaluate = x*x
    return evaluate
print('Calculated value is %d' %test_fnc(10))
Output:
Calculated value is 100
```

- c) Functions with multiple arguments (inputs), the order of the arguments can be altered during calling.

Function definition: Argument sequence

```
def test_fnc_2(a,b): #we change the order later
    evaluate = a*a + b
    return evaluate
print('Calculated value is %d' %test_fnc_2(a = 3, b = 4))
print('Calculated value is %d' %test_fnc_2(b = 4, a = 3))

Output:
Calculated value is 13
Calculated value is 13
```

3.2.1 Example: Function classifying the sign of a number

Another example on function definition where a list of numbers is classified based on whether they are positive, negative, or zero.

Listing 1: Higher Order Filter (HOF)

```
def pos_or_neg(x):      # definition starts with def
    if(x>0):
        return 'Positive'
    elif (x<0):
        return 'Negative'
    else:
        return 'Zero'   # end of function definition

for i in [10, 12, -1, 0, -5, 10]:
    print(pos_or_neg(i))
```

Output:

```
Positive
Positive
Negative
Zero
Negative
Positive
```

3.2.2 Example: Calculate circle area by defining a function

Let us define a function in Python that takes radius r as an input and calculate the corresponding area (A) of the circle.

Listing 2: Higher Order Filter (HOF)

```
import numpy as np
pi = 3.1414
x = np.random.randint(1, 100) # randomly generated numbers in between 1 to 100
r = np.random.randint(1, 10)  # randomly generated numbers in between 1 to 10
def square(x): # defining square function
    return x*x
def circle_area(r): # defining area function
    return pi*r*r
y = square(x)
area = circle_area(r)
print('Square of %d is %d' %(x,y))
print('Area for radius %d is %d' %(r,area))
```

Output:

```
Square of 67 is 4489
Area for radius 6 is 122
```

3.2.3 Example: Plot Mathematical Functions

Consider the below functions:

$$\text{fnc1} = f(x) = 5 - x^3 \quad (1)$$

$$\text{fnc2} = f(x) = 5 - (x - 0.5)^3 \quad (2)$$

We can define functions in Python and evaluate the functions for linearly spaced points ranging between -5 to 5. The exact code goes as follows:

Listing 3: Evaluating Mathematical Functions

```
import matplotlib.pyplot as plt

def fnc1(x):
    return 5 - x * x * x
def fnc2(x):
    return 5 - (x-0.5) * (x-0.5) * (x-0.5)

x = np.linspace(-5,5,100)
plt.plot(x,fnc1(x))
plt.plot(x,fnc2(x))
plt.xlabel("X")
plt.ylabel("Function value")
plt.legend(["fnc1","fnc2"])
plt.show()
```

3.3 Concepts of Higher Order Functions

Python allows functions to be used as another object or another piece of data. Precisely, function can be used as an argument (or parameter) in another function, as well as it can also be the returned value. That is, in Python:

- Function can be used as an argument
- It could be assigned to a variable using the assignment argument
- Function could be returned value of another function.

Let us define a function `test_fnc` and use the function to find the square of a number and print accordingly.

Listing 4: Higher Order Filter (HOF)

```
def test_fnc (x): # x is argument of the defined function test_fnc
    evaluate = x*x
    return evaluate
```

```
print('Calculated value is %d' %test_fnc(10))
print(test_fnc(10))
```

Output:

```
calculated value is 100
100
```

Interestingly, as Python handles function as an object as well, the defined function `test_fnc` can now be assigned to a variable (`M`), and subsequently, the variable inherits the underlying traits of the function `test_fnc`.

Listing 5: Higher Order Filter (HOF)

```
def test_fnc (x): # x is argument
    evaluate = x*x
    return evaluate
print('calculated value is %d' %test_fnc(10))
M = test_fnc # assignment to the variable M
print('calculated value is %d' %M(10))
```

Output:

```
calculated value is 100
calculated value is 100
```

Because the function in Python can be assigned to a variable, the function can be used as an argument in another function. For instance, the below example demonstrates the use of a function as an argument of another function (say, `higher_func`) through assigning the prior function to a variable `arg_func`. If a functional definition uses another function, as demonstrated below, the function is known as **Higher Order Function**.

Listing 6: Higher Order Filter (HOF)

```
def test_fnc (x): # x is argument
    evaluate = x*x
    return evaluate
M = test_fnc # assignment to the variable M
def higher_func(arg_fun, y): # higher order function
    final_value = arg_fun(y) * arg_fun(y)
    return final_value
```

```
higher_func(M, 3)
```

Output:

```
81
```

One interesting application of HOF would be to design filters for task-specific data processing. For instance, let us write a short code to select data from a text file and print.

Listing 7: Data Filter Design

```
import numpy as np
data = np.loadtxt('inp.txt')
print(len(data))
sliced = data[:5]
print(data[:5])
print(sliced)

def TestFilter(data):
    res = [] # Creating an empty list
    for d in data:
        if d>=50 and d<=60:
            res.append(d) #appending element to list
    return res

print('***** After Filtering *****')
filtered_data = TestFilter(data) # Function is treated as a data
print(len(filtered_data))
print(filtered_data[:5]) # Slicing, from begining upto 4 (not including 5)
```

Output:

```
1000
[52.157428  51.96758751 87.96353863 34.12761386 51.95632867]
[52.157428  51.96758751 87.96353863 34.12761386 51.95632867]
***** After Filtering *****
111
[52.15742800208103, 51.967587510202094, 51.95632866658776, 50.3288024879517,
 59.43570924203889]
```

3.3.1 Example HOF and List-comprehension: Map

The concept of Map in python takes a list and applies a function on the list element individually to obtain a new list. This new list is the return value of the callee function.

```
def CSE425_MAP(inp, f) :
    res = []
    for i in inp :
        res.append(f(i))
    return res

def sq(x) : return x * x
list_example = [5, 1, 3, 7, 4, 8, 9]
CSE425_MAP(list_example, sq)
```

Output:

```
[25, 1, 9, 49, 16, 64, 81]
```

3.4 Strings: Slicing

Python provides a slicing facility to slice a portion of a string or a list as needed. A few operations are as follows:

```
test_string = 'NorthSouth'
test_string2 = ' is Good'
print(test_string[0]) # Access 1st Character
print(test_string[0:2]) # Upto 2 but not including 2
test_string[0:2]
sliced_string_1 = test_string[:5] # From 0 to 4, not including 5
sliced_string_2 = test_string[5:] # From 6 to end, not including 5
sliced_string_3 = sliced_string_1 + sliced_string_2
print(sliced_string_1)
print(sliced_string_2)
print(sliced_string_3)
print() # Adding a linespace
print(test_string + test_string2) # Concaternation
```

Output:

```
N
No
North
South
NorthSouth
```

```
NorthSouth is Good
```

As known, Python also allows negative indexing, and accessing String elements using negative indexing goes as follows:

3.4.1 String operations

4 Control statements

The control statements in Python is very similar to *C*, and a few control statements are *if* statements, *for* loop, *while* loop. As it appears, the syntax of Python control statements is comparatively easier than C. For instance, Python does not have any use of curly brackets `{}`; instead, Python programming uses `:` and indentation as the syntax of the *for* loop. Precisely, `:` suggests the start of the block, and the content of the blocks is indented.

Sample code: Python

```
if condition < 3:
    print ("value_1")
else:
    print ("value_2")
```

Sample code: C

```
if (r < 3)
{
    printf("value_1\n");
}
else
{
    printf("value_2\n");
}
```

A snapshot of different control statements in Python is available as below.

4.1 If-else and Nested If-else

If Statements

```
x = 5
if(x<10):
    print("Python is easy")
else:
    print("Python is not easy")
```

colon (:) indicates starts the block, and no curly brackets { } as used in C. Statements inside the blocks are indented; it is mandatory.

Nested if else

```
score_1 = 91
if score_1>=93:
    print("Student's Letter Grade
          is A") # || of C is or in
                Python
elif (score_1 >=90) and
     (score_1<93):
    print("Student's Letter Grade
          is A-")
elif (score_1 >=87) and
     (score_1<90):
    print("Student's Letter Grade
          is B+")
else:
    print("Student's grade is
          something else")
```

4.2 While Loop

While Loop

```
i = 1
total = 0
while (i <= 10) :
    total = total + i
    print (total)
    i = i + 1
```

Output:

```
1
3
6
10
15
21
28
36
45
55
```

4.3 For Loop

Unlike the other loops, the **for loop** in Python does not have significant similarity to the for loop syntax as used in C-programming. Precisely, **for loop** in python considers the list to iterate over the elements included in the list.

For Loop

```
list_1 = range(1, 10, 2)
for i in list_1:
    print(i)
```

```
data = [4, 6, 9, 10, 15, 17, 20, 22]
```

```
for i in range(1, 8, 2):
    print(data[i])
```

Output:

```
6
10
17
22
```

The **range** function allows to start a loop iteration from any desired value and ends so. Additionally, the looping can also accommodate spacing between two successive iterations.

5 Data structures in Python

Unlike other programming languages, Python offers a number of built-in data structures, and the data types are: i) Lists, ii) Tuple, iii) Strings, iv) Sets, v) Dictionaries.

5.1 Lists

In Python, *list* is a sequenced collection of objects (number, names, strings etc.), and is enclosed using [] with comma separated entries.

```
list_1 = [1, 2, 3, 4, 5]
print(list_1)
Output: [1, 2, 3, 4, 5]
```

Lists can include any types of elements in Python:

```
list_2 = ['a', 'b', 'c']
print(list_2)
Output: ['a', 'b', 'c']
```

A list may include multiple types of elements, as shown below.

```
list_3 = [1, 2, 3, 4, 5, 'a', 'b', 'c']
print(list_3)
Output: [1, 2, 3, 4, 5, 'a', 'b', 'c']
```

A list of can be repeated up to a desired size with a default set of elements:

```
list_4 = 4 * [1, 2, 3]
print(list_4)
Output: [1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3] # repeated 4 times

list_5 = 4 * ['a', 'b', 'c']
print(list_5)
Output: ['a', 'b', 'c', 'a', 'b', 'c', 'a', 'b', 'c', 'a', 'b', 'c']
```

New elements to any list can be appended at the end of a list by the `list_name.append()` command.

```
list_1 = ['a', 'b', 'c']
list_1.append('d')
print(list_1)
Output: ['a', 'b', 'c', 'd']
```

Element indexing in any list starts from zero. Also, negative indexing is allowed. For instance, `list_1[-1]` returns the last element of a given list.

```
list_1 = [2, 4, 5, 19, 7]
x = list_1[3]
print(x)
Output:19
```

It is possible to insert desired elements in specific positions of a list:

```
list_1 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
list_1.insert(3, 'a')
print(list_1)
Output: [1, 2, 3, 'a', 4, 5, 6, 7, 8, 9, 10]
```

`list_1.remove('a')`: Specific element (for instance, 'a' here) from the defined list can be removed as follows:

```
list_1 = [1, 2, 3, 'a', 4, 5, 6, 7, 8, 9, 10]
list_1.remove('a')
print(list_1)
Output: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

`list.reverse()`: An existing list can be reversed using the inbuilt list properties `list.reverse()`

```
list_1 = [1, 2, 3, 'a', 4, 5, 6, 7, 8, 9, 10]
print(list_1)
list_1.reverse()
list_1
Output: [10, 9, 8, 7, 6, 5, 4, 'a', 3, 2, 1]
```

All the elements of a list can be accessed using a for loop:

```
list_1 = [2, 5, 7, 9]
for i in list_1:
    print(i)    # indentation is necessary
```

```
Output:
2
5
7
9
```

Copy vs. reference: It is important to remember the difference between **copy** and **reference** to a list. When a list is defined under a variable such as `list_1`, it acts as a reference to the memory location. The variable `list_1` contains a reference to the `list_1`, not the content of it.

For instance, in the below example, `list_1` and `list_2` both represent the same list in the memory, and they denote the memory reference, not the actual content of the list. As a result, any change in either of the lists is reflected in both `list_1` and `list_2`.

```
list_1 = [1, 2, 3, 4, 5]
list_2 = list_1 # reference, not the content, is passed to list_2
print(list_1)
list_1.append(7) # appending 7 to the list_1
print(list_1)
print(list_2)
```

Output: # Change in list_1 is reflected in list_2

```
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5, 7]
[1, 2, 3, 4, 5, 7]
```

Alternative ways for list formation: Interestingly, lists could be copied in alternative many ways. As in the below example, lists are copied, and the change in the original list through an append option is not reflected in the copied version of the list.

```
list_1 = [1, 2, 3, 4, 5]
list_2 = list_1[:] # Copying the content of list_1 to list_2
list_3 = list_1.copy()
print(list_1)
print(list_2)
print(list_3)
list_1.append(7) # appending 7 to the list_1
print(list_1)
print(list_2)
print(list_3)
```

Output:

```
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5, 7]
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5]
```

Slicing: In Python, list can be partially copied as well through the slicing concept. The list slicing copies specific indices and form another list.

```
list_1 = [1, 2, 3, 4, 5, 'a', 'b', 'c']
print(list_1)

list_2 = list_1[1:6]
print(list_2)
```

Output:

```
[1, 2, 3, 4, 5, 'a', 'b', 'c']
```

```
[2, 3, 4, 5, 'a']
```

In the above example, slicing starts from 1st position and ends at 6th position. As the indexing starts from 0, indexing ends at 5 in this example.

0	1	2	3	4	5	6	7
1	2	3	4	5	a	b	c

If the slicing indexes are not provided, Python considers default values to continue with the operation. The below examples clearly manifest how the slicing is done with the default indexes included in the syntax.

```
list_1 = [1, 2, 3, 4, 5, 'a', 'b', 'c']
print(list_1)

list_2 = list_1[:5] # keep elements upto 4th element; 5th not included.
print(list_2)

list_3 = list_1[5:] # keep all elements starting from the 5th element
print(list_3)
```

Output:

```
[1, 2, 3, 4, 5, 'a', 'b', 'c']
[1, 2, 3, 4, 5]
['a', 'b', 'c']
```

It is possible to select elements from any desired interval. Precisely, introducing a third argument in the list slicing, any desired elements can be selected from the list.

```
list_1 = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
even_no = list_1[0:len(list_1):2]
print(even_no)
```

Output:

```
[0, 2, 4, 6, 8]
```

A list can include multiple lists as elements— that is, nested list is possible as observed from the below example.

```
nested_list = [[1, 2], [3, 4], [5, 6]]
print(nested_list)
```

Output:

```
[[1, 2], [3, 4], [5, 6]]
```

5.2 Tuple

By definition, tuple is an ordered collected of objects with provision that elements could be repeated. In Python, once tuple is defined, the tuple cannot be extended, modified. Precisely, no new element can be added or removed from the tuple.

```
tuple_1 = (1, 2, 5, 7)
print(tuple_1)
print(tuple_1[0]) # accessing element 1
```

Output:

```
(1, 2, 5, 7)
1
```

As already state, elements of a tuple cannot be removed, or new element can be added.

```
tuple_1 = (1, 2, 5, 7)
tuple_1.remove(5)
```

Output:

```
AttributeError: 'tuple' object has no attribute 'remove'
```

A tuple can include list as an element, and the list can be modified. However, the tuple elements cannot be extended, modified or removed. That is, the list that is included in the tuple cannot be removed.

```
tuple_1 = (2, 4, 6, []); # an empty list as tuple element
print(tuple_1)
print(tuple_1[3])

tuple_1[3].append(10);
tuple_1[3].append(12);
print(tuple_1[3])
print(tuple_1)
```

Output:

```
(2, 4, 6, [])
[]
[10, 12]
(2, 4, 6, [10, 12]) # empty list is appended
```

Multiple outputs can be placed in a Tuple as the return value of a function, as demonstrated in the below example.

```
def quotient_remainder_tuple(x, y):
    quot = x // y # Floor division, returns floor(x/y)
    rem = x % y
    return (quot, rem)
```

```
(quotient, remainder) = quotient_remainder(12,7)
returned_tuple = (quotient, remainder)
print("Quotient is ",quotient)
print("Remainder is ", remainder)
print("Returned Tuple is ", returned_tuple)
```

Output:

```
Quotient is 1
Remainder is 5
Returned Tuple is (1, 5)
```

5.3 Strings

Similar to other programming language, Python also provides string data types. Each character in Strings can be separately accessed. A few example operations on strings in python are as follows:

```
string_1 = 'THIS IS CSE425 CLASS'
display(string_1)
display(string_1[0])
display(string_1[4])
display(len(string_1))
```

Output:

```
'THIS IS CSE425 CLASS'
'T'
' '
20
```

A simple for loop is sufficient to access all the elements of a string:

```
string_1 = 'TEST'
for i in string_1:
    print(i)
```

Output:

```
T
E
S
T
```

Negative indexing can also be used to access the characters of a string, as demonstrated below:

```
string_1 = 'THIS IS CSE425 CLASS'
print(string_1[-1])
print(len(string_1))
```

```
print(string_1[-20])
print(string_1[-5:-1]) # From -5 index upto -2, doesn't include -1
```

5.4 Sets

The set data type is a collection of unordered unique elements. So, no element can repeat in a set, and the elements may not appear following the order the set is defined.

```
set_1 = {1, 2, 3}
set_2 = {'a', 'b', 'c'}
print(set_1)
print(set_2)
```

Output:

```
{1, 2, 3}
{'a', 'c', 'b'} # unordered appearance of elements
```

As the order is not maintained in the set definition, accessing a specific set element requires mentioning it precisely. By definition, a set includes only unique elements. It is also possible to remove elements from a set, as elaborated below.

```
set_1 = {1, 2, 3}
set_2 = {'a', 'b', 'c', 'b'}
print(set_1)
print(set_2)
```

```
set_2.remove('c')
print(set_2)
```

Output:

```
{1, 2, 3}
{'a', 'c', 'b'}
{'a', 'b'}
```

Additional elements can be added to an existing list:

```
set_1 = {1, 2, 3}
print(set_1)
```

```
set_1.add('c')
print(set_1)
```

Output:

```
{1, 2, 3}
{1, 2, 3, 'c'}
```

5.5 Dictionaries

Dictionary is another **data type** available in Python. Dictionaries are very similar to sets, but each element in dictionary comprises of a pair of items denoting the *key* and value, respectively. In dictionary, no **key** appears more than once.

```
dict_1_cse_173 = {'S1' : 5, 'S2' : 6, 'S3' : 7} #S1 is key, 5 is value
print(dict_1_cse_173)
```

Output

```
{'S1': 5, 'S2': 6, 'S3': 7}
```

Additional key and value could be included to the existing dictionary.

```
dict_1_cse_173 = {'S1' : 5, 'S2' : 6, 'S3' : 7} #S1 is key, 5 is value
print(dict_1_cse_173)
```

```
dict_1_cse_173['S4'] = 10
print(dict_1_cse_173)
```

Output

```
{'S1': 5, 'S2': 6, 'S3': 7}
{'S1': 5, 'S2': 6, 'S3': 7, 'S4': 10}
```

For an iterative access to the keys and values of a dictionary, we can use **for loop**.

```
dict_1_cse_173 = {'S1' : 5, 'S2' : 6, 'S3' : 7} #S1 is key, 5 is value
for i in dict_1_cse_173:
    print(i)
    print(dict_1_cse_173[i])
```

Output:

```
S1
5
S2
6
S3
7
```

Both **key** and **value** can be printed together as given below.

```
dict_1_cse_173 = {'S1' : 5, 'S2' : 6, 'S3' : 7} #S1 is key, 5 is value
for i, j in dict_1_cse_173.items():
    print('Key {} is {}'.format(i, j))
```

Output:

```
Key S1 is 5
Key S2 is 6
Key S3 is 7
```

The above example includes `format()` for the substitutions of the values to be placed in the desired position.

Syntax : `{ }.format(value)`

6 List Comprehension

```
import numpy as np
data = np.random.randint(0, 10, size=25) #25 random integer from 0 to 10
print(data)
print(len(data))
```

Output:

```
[3 9 6 9 1 9 4 2 6 7 8 8 9 2 0 6 7 8 1 7 1 4 0 8 5]
25
```

We can use the **list comprehension** approach to select the desired data (even data, or divisible by 4) using a single line code as demonstrated below:

Listing 8: List Comprehension

```
all_even = [i for i in data if (i % 2) == 0]
print(all_even)

divisible_by_4 = [i for i in data if (i % 4 == 0)]
print(divisible_by_4)
```

Output:

```
[6, 4, 2, 6, 8, 8, 2, 0, 6, 8, 4, 0, 8]
[4, 8, 8, 0, 8, 4, 0, 8]
```

We also can create a combination of two different ways to filter/process the existing data. For instance, we want to find the square of all the numbers in data that are divisible by 4.

```
task = [i**2 for i in [j for j in data if (j % 4 == 0)]]
print(task)
```

Output:

```
[16, 64, 64, 0, 64, 16, 0, 64]
```

The task can be achieved using a single-line code that uses the concept of list comprehension.

Matrix generation using list-comprehension: Matrix of elements calculated using other mathematical operations can be formed with the concept of list-comprehension. A sample code snippet that forms a matrix using the list comprehension. Here, we are nesting one list comprehension to the other list comprehension.

```
matrix = [  
    [(a+1)*(i+1) for a in range(3)]  
    for i in range(3)  
] # nested list comprehension  
print(matrix)
```

Output:

```
[[1, 2, 3], [2, 4, 6], [3, 6, 9]]
```

6.1 Quicksort algorithm using list comprehension

With a few lines code that include list-comprehension approach, it is possible to implement the quicksort algorithm in Python ¹.

Listing 9: Quick Sort in Python

```
def quicksort_list(input_array): # functional definition starts with def  
    if len(input_array) <= 1:  
        return input_array  
    pivot_element = input_array[len(input_array) // 2] # floor value is obtained  
    left = [x for x in input_array if x < pivot_element] # list comprehension  
    middle = [x for x in input_array if x == pivot_element]  
    right = [x for x in input_array if x > pivot_element] # list comprehension  
    return quicksort(left) + middle + quicksort(right) # divide and conquer  
  
print(quicksort_list([3, 3, 3, 6, 8, 10, 1, 2, 1, 10, 15]))
```

Output:

```
[1, 1, 2, 3, 3, 3, 6, 8, 10, 10, 15]
```

7 Object Oriented Programming (OOP)

By definition, an OOP paradigm designs programs for a given task considering the concepts of objects. Like other OOP paradigms, Python implements OOP by defining classes.

7.1 Objects

Objects are data abstraction that captures data attributes as well as an interface in the form of methods.

¹<https://codereview.stackexchange.com/>

7.2 Class Definition

OOP: Class

```
class basicMath:
    def subtraction(self, a, b):
        return a-b
    def product(self, a, b):
        return a*b
    def addition(self, a, b):
        return a+b
    def division(self, a, b):
        return a/b

m = basicMath() # Creating an instance of the Object, and m inherits all
                # the functions and behaviors
x1 = m.subtraction(8, 6)
x2 = m.product(8, 6)
x3 = m.addition(8, 6)
x4 = m.division(8, 6)
print("Subtraction is",x1)
print("Product is",x1)
print("Addition is",x1)
print("Division is %.2f" %x4)
```

Output:

```
Subtraction is 2
Product is 2
Addition is 2
Division is 1.33
```

```
class Parent():
    def test_fnc1(self):
        print('test-one print in Parent')
    def test_fnc2(self):
        print('test-two in Parent')

class Child(Parent):
    pass # not to add/define anyother methods to class

value1 = Parent()
value1.test_fnc1()

value2 = Child()
value2.test_fnc1()
```

Output:

```
test-one print in Parent
test-one print in Parent
```

7.3 Define Classes and Objects in Python

Like C++ and Java, object-oriented programming can be performed with Python.

- It is possible to perform object-oriented programming using Python.
- We can use `class` to construct object

7.4 `__init__()` in Python: Constructor for a Class

The class definition syntax is sufficient to define an object class, but not of much practical use. Instead,

- All the classes in Python use the built-in `__init__()` function.
- Values or operations allowed by the object are all decided by the `__init__()` function.

Let's define an object class with name `cse425student` with properties `name`, `id`, `sec`, `grade`

`__init__()` function

```
class cse425student:
    def __init__(self, name, id, sec, grade):
        self.name = name
        self.id = id
        self.sec = sec
        self.grade = grade

s1 = cse425student("karim", 23, 1, "A")

print(s1.name)
print(s1.id)
print(s1.sec)
print(s1.grade)
```

Output:

```
karim
23
1
A
```

8 Concurrency and Parallelism

8.1 Multiprocessor Architecture

Most modern computers have multiprocessors where more than one task could be executed in a concurrent fashion. That is, the concurrent execution of programs is mostly supported by modern computing hardware which often has chips hosting multiple processors. So, if the software is not adjusted to make more use of the multiple processors then the concurrent hardware remains unused.

8.2 Concurrency

Concurrency suggests the execution of multiple activities simultaneously without any interference between the activities. For instance,

- Multiple computers in a network
- Multiple applications running on the same computer

are all instances of concurrency. The design of **Web browsers** is heavily dependent on the concept of concurrency, where the browser performs different functions (tasks) at the same time. For instance, a browser sends and receives data from Web servers, renders texts and images on the screen, and reacting users' actions, etc. All modern programming languages offer concurrency, either by default or through additional libraries. Concurrency may occur at alternative levels:

- Machine instruction level: Two or more machine instructions may execute concurrently
- Statement level: Two or more statements may execute simultaneously
- Subprogram level: Two or more subprograms may execute concurrently
- Program level: Two or more programs may be concurrently executed

Among the four types, Statement level and Subprogram level concurrency are dealt with at the time of programming language design and are of our interest.

```
import time
def mt():
    for i in range(5):
        print (i)
        time.sleep(1)
mt()
```

We can use **multiprocessing** module and as shown in the example below ²:

```
import time
from multiprocessing import Pool

def main():
    p = Pool(processes=5) # start 5 worker processes
    result = p.map(some_func, range(5))

def some_func(i):
    print(i)
    time.sleep(1)

if __name__ == '__main__':
    main()
```

8.3 Parallelism

9 Numerical Simulation

9.1 Review of Linear Algebra

Python is also widely used for numerical computation, which often requires extensive use of linear algebra concepts.

²<https://stackoverflow.com/questions/54550888>

9.1.1 Vector

The i^{th} element in a vector is accessed as x_i , where x is the vector and the index i starts from 0. A few simple examples are as:

```
import numpy as np
x = np.array([2, 4, 35])
print(x)
print(x[0])
```

Output:

```
[ 2  4 35]
2
```

```
vector_1 = 10 * np.ones(4, dtype = int)
print(vector_1)
```

Output:

```
[10 10 10 10]
```

Basic vector operation such as vector addition, multiplication of a vector by scalar etc. are as follows:

```
vector_1 = 10 * np.ones(4, dtype = int)
print(vector_1)
vector_2 = 5 * np.ones(4, dtype = int)
print(vector_2)
c = vector_1 + vector_2
print(c)
```

Output:

```
[10 10 10 10]
[5 5 5 5]
[15 15 15 15]
```

Vector + scalar

```
vector_3 = 4 * np.ones(5, dtype = int)
vector_4 = vector_3 + 1
print(vector_3)
print(vector_4)
```

Output:

```
[4 4 4 4 4]
[5 5 5 5 5]
```

References

- [1] David M Beazley. *Python essential reference*. Addison-Wesley Professional, 2009.
- [2] Guido Van Rossum and Fred L Drake Jr. *Python tutorial*, volume 620. Centrum voor Wiskunde en Informatica Amsterdam, The Netherlands, 1995.